1. Objective

Lab 6 decomposed the backend into three independently runnable microservices - User, Product and Order - communicating directly over a Docker network, with the API Gateway introduced only as a concept. This lab makes that concept real: build an actual **API Gateway** as the single entry point for clients, make the gateway's knowledge of service locations **configurable rather than hard-coded** (a lightweight form of service discovery), and finally **deploy the containerized system to the cloud** so it is reachable over the internet, not just localhost.

Flow for this lab:

Client / Postman → API Gateway (public, cloud-hosted) → Service Registry / Config → User / Product / Order Services (containers) → MongoDB Atlas

2. Part A - Build a Real API Gateway

Replace direct client-to-service calls with a single gateway entry point. The gateway contains **no business logic** - it only routes requests and handles cross-cutting concerns.

- Create a new **api-gateway** service in the same tech stack as your Lab 6 services (e.g. Express + http-proxy-middleware, or the Spring Boot / Node equivalent you used).
- Route incoming paths to the correct backend service: /users/* → User Service, /products/* → Product Service, /orders/* → Order Service.
- Add a gateway health-check endpoint, e.g. GET /health, that reports the gateway is running.
- Add basic request logging at the gateway (method, path, target service, response status).
- Add centralized error handling: if a target service is unreachable, the gateway returns a clear 502/503 response instead of hanging or crashing.
- Only the gateway's port should be exposed externally in Docker Compose - User, Product and Order should be reachable only from inside the Docker network, not directly from outside.

> Discussion question (answer briefly in your README): **Why introduce an API Gateway instead of letting clients call each service directly?** Think about a single entry point, hiding internal structure, and centralizing concerns like logging and error handling.

Concrete example: one gateway route

Primary example uses **Node / Express with http-proxy-middleware** (use the equivalent in your own stack if different). Follow one request end to end:

```
Client  --  GET /users/101  -->  API Gateway
API Gateway  --  GET http://user-service:3001/users/101  -->  User Service
```

```
const express = require("express");
const cors = require("cors");
const { createProxyMiddleware } = require("http-proxy-middleware");
const app = express();
app.use(cors());   // only needed for browser (React) clients

app.use(createProxyMiddleware({
  pathFilter: "/users",
  target: process.env.USER_SERVICE_URL,   // e.g. http://user-service:3001
  changeOrigin: true,
  on: { error: (err, req, res) => {
    res.writeHead(503, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ error: "User Service unavailable" }));
  } }
```

```
}));
```

*Repeat the same block for /products and /orders using their own *_SERVICE_URL values.* ***CORS note:*** *if a browser-based React client calls the cloud API, CORS may need to be configured at the gateway/API layer (as with cors() above). This is a brief note, not a separate topic.*

Hands-on exercise: gateway failure test

Make the 502/503 behaviour an explicit practical test and record each step in Postman:

1. Start all services (gateway, User, Product, Order).
2. Call GET /users/101 through the gateway and confirm a normal 200 response.
3. Stop the User Service.
4. Repeat the same request through the gateway.
5. Verify the gateway returns an appropriate 502/503 response - it must not hang or crash.
6. Restart the User Service and repeat the request to verify recovery.

This demonstrates why an API Gateway provides centralized error handling.

3. Part B - Service Discovery (Configuration-Based)

The gateway must not hard-code service locations inside route-handling code. Externalize them into configuration so locations can change without touching application code - this is the lightweight, practical form of service discovery for this lab.

- Define a small service registry as environment variables or a config file, e.g.: USER_SERVICE_URL, PRODUCT_SERVICE_URL, ORDER_SERVICE_URL.
- The gateway reads this configuration at startup and uses it to build its routing table - route definitions should reference the config values, not literal hard-coded URLs.
- Prove it works: change a service's URL/port only via configuration (no code change) and confirm the gateway routes correctly to the new location.
- In your README, briefly contrast this static/config-based approach with **dynamic** service discovery (e.g. Consul, Eureka, Kubernetes DNS) - what would a dynamic registry add that a static config file cannot?

Local vs cloud service URLs

In Lab 6, Docker services reached each other by name (e.g. http://user-service:3001). On a cloud platform the addressing mechanism may be different:

Environment	Service URL Example
Local Docker Compose	http://user-service:3001
Cloud internal service	Platform-provided internal URL / hostname
Public API	https://<gateway-public-url>

Key principle: **service locations must be configuration, not application code.** The same application image should run locally and in the cloud simply by changing environment variables.

4. Part C - Cloud Deployment

Deploy the containerized system so the API Gateway is reachable over the internet. Your MongoDB is already cloud-hosted on Atlas since Lab 4 - this step moves the application layer to the cloud too. Cloud deployment is an **extension of your working local system, not a new application**.

- Choose a cloud platform that can run your Docker containers, e.g. **Render**, **Railway**, **Fly.io**, or a free-tier VM/App Service on AWS, GCP or Azure.
- Deploy the API Gateway and the three microservices (User, Product, Order) as containers on the chosen platform.
- Configure environment variables for the cloud environment (service URLs, MongoDB Atlas connection string) using the same config-driven approach from Part B - do not hard-code cloud values either.
- Confirm the deployed API Gateway has a public URL reachable from outside your machine.
- Re-run your Postman tests against the **public gateway URL** (not localhost) to confirm the full flow works in the cloud: gateway → services → MongoDB Atlas.

> If your platform's free tier cannot host all three services plus the gateway at once, deploy the gateway and at least one complete, working chain (e.g. gateway + User Service, or gateway + all three if resources allow) and clearly document what was deployed and any limitation you hit.

Cloud deployment sequence

1. Verify the local Lab 6 system works.
2. Build the service images.
3. Select a cloud platform.
4. Push / build the images on the platform.
5. Deploy the services.
6. Configure environment variables.
7. Configure MongoDB Atlas.
8. Deploy the API Gateway.
9. Obtain the public gateway URL.
10. Test through the public URL.

MongoDB Atlas and environment configuration

```
MONGODB_URI=<Atlas connection string>
USER_SERVICE_URL=<service URL>
PRODUCT_SERVICE_URL=<service URL>
ORDER_SERVICE_URL=<service URL>
```

- Do **not** commit passwords or connection strings to GitHub.
- Use the platform's cloud environment variables / secrets.
- Verify that the deployed services can connect to MongoDB Atlas.

5. Architecture

Layer	Responsibility	Reachable From
Client / Postman	Sends requests to one public address	Internet
API Gateway	Routes /users, /products, /orders; logging; error handling	Internet (public)
User / Product / Order Services	Own business logic and own data	Docker network only
MongoDB Atlas	Persistent storage per service	Services (via connection string)

Your architecture diagram (in the README) should show this same layering: Client/Postman → API Gateway → User/Product/Order Services → MongoDB Atlas, with the Docker network boundary drawn around the three services.

Local vs cloud architecture

Layer	Local (Docker Compose)	Cloud
Entry	Postman / React	Internet client
Gateway	API Gateway container	Public API Gateway
Network	Docker network	Internal / private network
Services	User, Product, Order (service names)	User, Product, Order (platform internal URLs)
Database	MongoDB	MongoDB Atlas

What stays the same: *application code, routes, API contract.* ***What changes:*** *where it runs, how service URLs are resolved, and public exposure of the gateway.*

6. Gateway Endpoints

Gateway Path	Routed To	Example
GET /users, /users/{id}	User Service	GET /users/101
POST /users, PUT /users/{id}, DELETE /users/{id}	User Service	POST /users
GET /products, /products/{id}	Product Service	GET /products/501
POST /products, PUT /products/{id}, DELETE /products/{id}	Product Service	POST /products
POST /orders, GET /orders, GET /orders/{id}	Order Service	POST /orders
GET /health	Gateway itself	Health check, no proxying

7. Step-by-Step Procedure

- **Step 1 —** Confirm your Lab 6 User, Product and Order services still run correctly together via Docker Compose.
- **Step 2 —** Design the gateway's routing table: which path prefixes map to which service.
- **Step 3 —** Scaffold the api-gateway service in your existing tech stack.
- **Step 4 —** Externalize service locations into environment variables / a config file (service discovery layer).
- **Step 5 —** Implement reverse-proxy routes in the gateway for /users, /products and /orders using that config.
- **Step 6 —** Add a GET /health endpoint, request logging, and centralized 502/503 error handling for unreachable services.
- **Step 7 —** Add api-gateway to docker-compose.yml on the same network; expose only the gateway's port externally.
- **Step 8 —** Test locally in Postman and complete the gateway failure test (stop User Service, expect 502/503, restart, verify recovery).
- **Step 9 —** Follow the cloud deployment sequence in Part C: deploy the services and gateway, set the cloud environment variables (service URLs, MongoDB Atlas URI), and obtain the public gateway URL.
- **Step 10 —** Test the deployed system using the gateway's public URL from Postman, confirming the full gateway to service to MongoDB Atlas flow works over the internet.
- **Step 11 —** Update README.md with the architecture diagram, the discovery/config approach, deployment steps, the public gateway URL, and troubleshooting notes.

8. Submission Requirements (What to Submit)

- Source code and Dockerfile for the api-gateway, alongside the existing User, Product and Order services from Lab 6.
- Updated docker-compose.yml including the gateway, with only the gateway's port exposed externally.
- Service registry / configuration (env vars or config file) showing service locations are not hard-coded in route-handling code. Secrets and the Atlas connection string must not be committed.
- Postman collection/screenshots showing: gateway-routed calls to all three services, the GET /health check, the gateway failure test (502/503 when User Service is stopped, and recovery after restart), and the same tests re-run against the public cloud URL.
- Evidence of cloud deployment: platform used, public gateway URL, and configuration steps (screenshots of the deployment dashboard/CLI output).
- Updated README.md containing: the architecture diagram, the API Gateway discussion answer, the static-vs-dynamic service discovery discussion, deployment steps, and troubleshooting notes.
- A short written reflection (5-8 lines) on what the gateway and cloud deployment changed about how the system is used and operated, compared to Lab 6.

9. Learning Outcomes

By the end of this lab, students will be able to: explain why an API Gateway is used as a single entry point in a microservices system; implement gateway routing, logging and centralized error handling; externalize service locations into configuration as a lightweight form of service discovery, and explain how this differs from dynamic service discovery; and deploy a containerized multi-service application to a cloud platform so it is reachable over the internet.

10. Final Checklist

API Gateway	Service Discovery	Cloud Deployment	Evidence
api-gateway service created	Service URLs in env vars/config	Cloud platform chosen	Postman: gateway routes
Routes to User/Product/Order	No hard-coded URLs in route code	Gateway + services deployed	Postman: /health check
GET /health implemented	Config change proven without code change	Env vars set; Atlas connected, no secrets committed	Failure test: 502/503 + recovery
Request logging added	Static vs dynamic discussed in README	Public gateway URL reachable	Cloud deployment screenshots
502/503 on unreachable service	Local vs cloud URLs understood	Postman re-tested on public URL	Updated README.md

Important: Keep the resource design, endpoints and status codes from Lab 6 unchanged. This lab adds a gateway, configuration-based service discovery and cloud deployment - it does not redesign the microservices themselves.